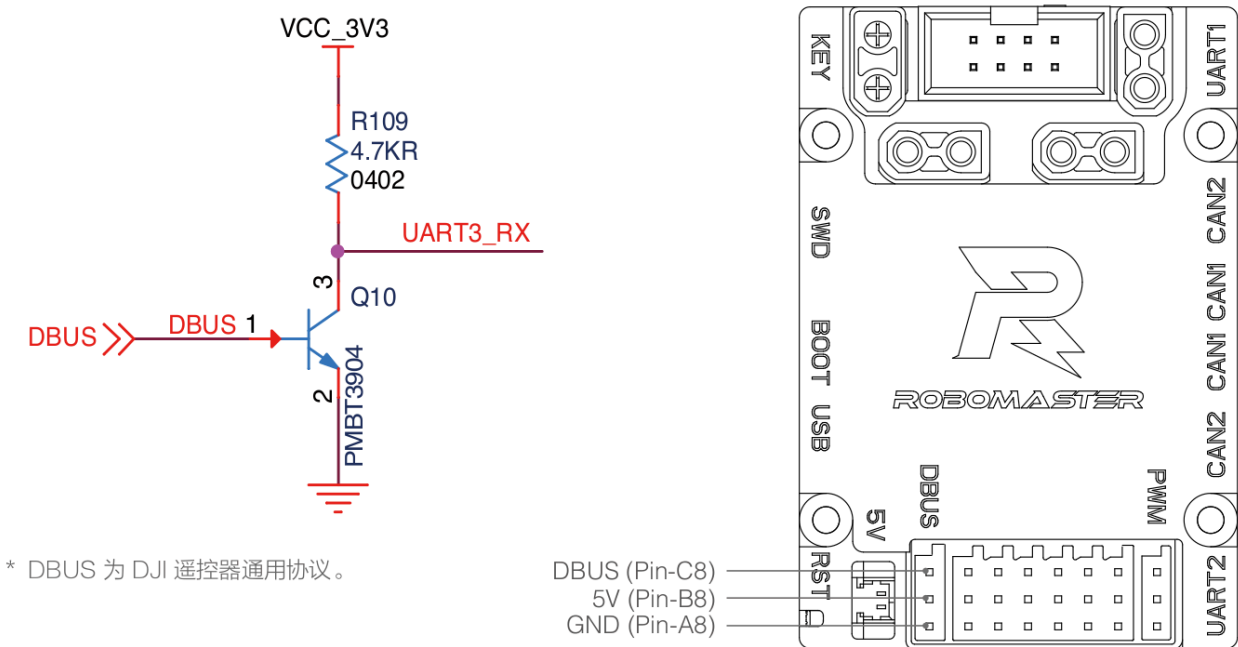


1. 接线

以RoboMaster C型开发板和天地飞ET08A遥控器为例

DBUS 接口

开发板集成 1 路 DBUS 接口，与 PWM 接口共用一个连接器，其接口原理图如下所示。DBUS 信号经反相电路后连接到 STM32 的 UART3，波特率一般设置为 100kbps。

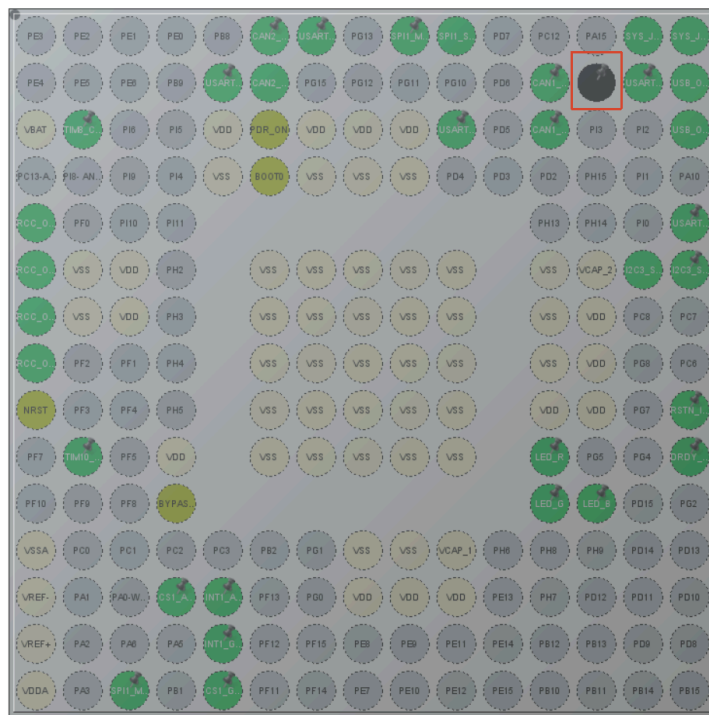
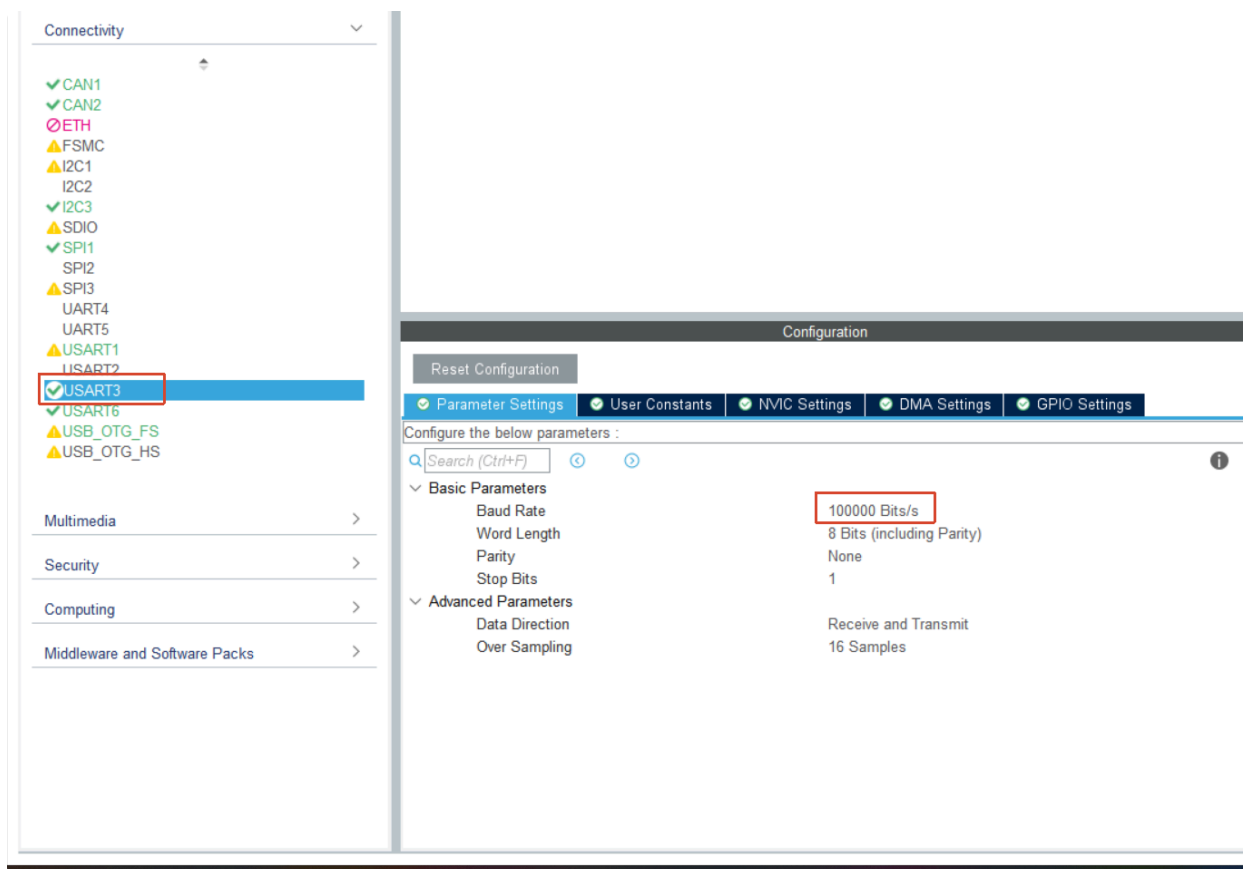


我们需要将遥控器接收机的SBUS接口接到C板上的DBUS接口，注意线序一定要接对。

2. CubeMX配置

由手册得知，单片机通过UART3串口接受遥控器的数据，我们需要在CubeMX中打开UART3的RX引脚，为PC11，并将波特率改为100kbps，开启串口DMA

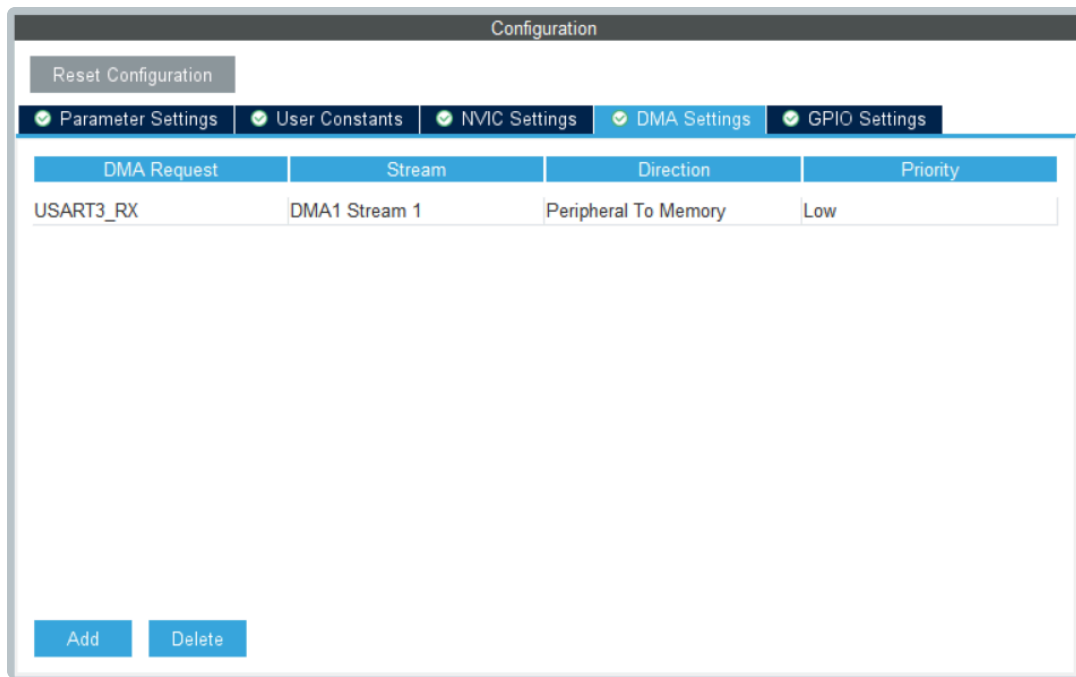
DBUS 接口	UART3_RX	PC11
---------	----------	------



UFBGA176+25 (Top view)



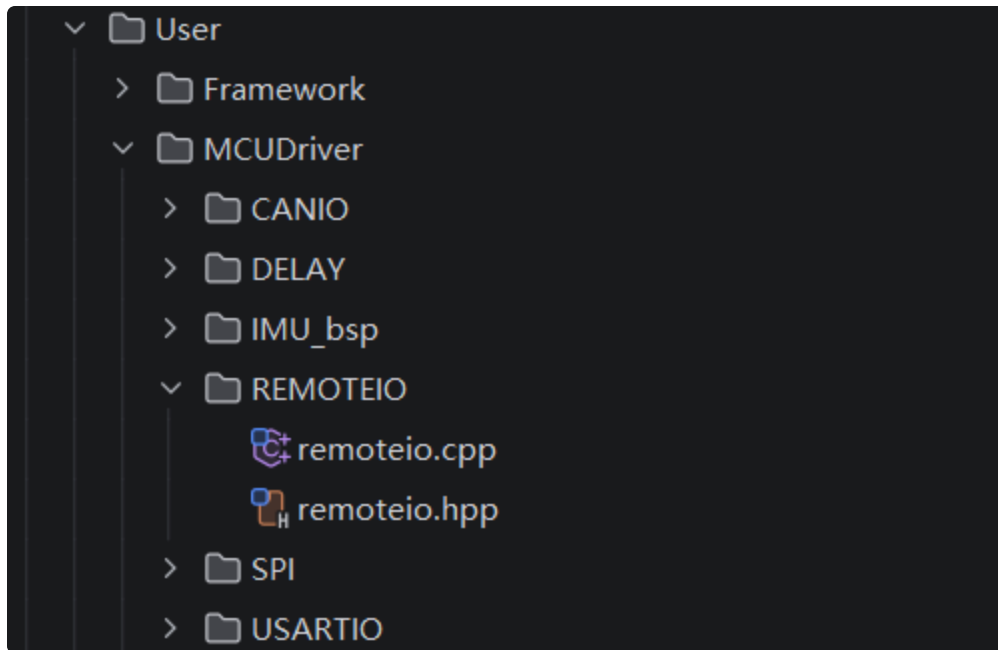
只需开启接收的DMA通道即可



配置完后生成代码

3. 代码移植

参考2026赛季无人机的云台代码，需要将 /User/MCUDriver/REMOTEIO 下的文件粘贴到自己的工程，并删除代码中自己工程中不存在的头文件的引用



在 main.cpp 里引用 remoteio.hpp ，并在主函数的初始化里面调用函数

```
REMOTEIO_Init(sbus_rx_buf[0], sbus_rx_buf[1], SBUS_RX_BUF_NUM);
```

在 /Core/Src/stm32f4xx_it.cpp 文件中引用找到

```

/**
 * @brief This function handles USART3 global interrupt. */void
USART3_IRQHandler(void)
{
    /* USER CODE BEGIN USART3_IRQn 0 */

    /* USER CODE END USART3_IRQn 0 */
    HAL_UART_IRQHandler(&huart3);
    /* USER CODE BEGIN USART3_IRQn 1 */

    /* USER CODE END USART3_IRQn 1 */
}

```

并在其中插入遥控器接收数据处理函数

```
REMOTE_UartIrqHandler();
```

如下

```

/**
 * @brief This function handles USART3 global interrupt.
 */
void USART3_IRQHandler(void)
{
    /* USER CODE BEGIN USART3_IRQn 0 */

    /* USER CODE END USART3_IRQn 0 */
    HAL_UART_IRQHandler(&huart3);
    /* USER CODE BEGIN USART3_IRQn 1 */
    REMOTE_UartIrqHandler();
    /* USER CODE END USART3_IRQn 1 */
}

```

然后可以在 `subs_to_rc()` 函数中打印，看是否成功接收并正确解码遥控器的数据，例如

```
static void subs_to_rc(volatile const uint8_t *sbus_buf, RC_ctrl_t *rc_ctrl)
{
    rc_ctrl->sbus_status.frame_lost = (flags & SBUS_FLAG_FRAME_LOST) ? 1 : 0;

    // 收到有效帧，重置软件看门狗（无论 failsafe 状态，接收到帧说明链路通畅）
    rc_ctrl->rc_live_cnt_WFLY          = RC_OFFLINE_TIMEOUT;
    rc_ctrl->sbus_status.rc_offline = 0;

    RC_DataHandle(rc_ctrl);

    //打印遥控器原始数据

    // usart_printf("%d, %d, %d, %d, %d, %d, %d\r\n", sbus_buf[0], sbus_buf[1], sbus_buf[2], sbus_buf[3], sbus_buf[4], sbus_buf[5], sbus_buf[6]);

    // 打印遥控器解算后数据

    // usart_printf("%d, %d, %d, %d, %d, %d, %d, %d\r\n",
    // rc_ctrl->rc.ch[0], rc_ctrl->rc.ch[1], rc_ctrl->rc.ch[2], rc_ctrl->rc.ch[3],
    // rc_ctrl->rc.ch[4], rc_ctrl->rc.ch[5], rc_ctrl->rc.ch[6], rc_ctrl->rc.ch[7]);
    /* 分别对应 右横 右竖 左横 左竖 SD SC SB SA */
    /* 前四个值在回中时应该为0（若不是则去修改.h文件中原始数据的回中值），后四个值在回中时应该为1024 */
}
```

通过调用 `RC_ctrl_t` 结构体的实例 `rc_ctrl` 来使用遥控器的参数

```
typedef struct
{
    struct
    {
        /* 遥控器摇杆和拨杆 */
        int16_t ch[8];
        char s[4];
    } rc;

    SBUS_Status_t sbus_status;
    uint16_t      rc_live_cnt_WFLY; /*!< 在线倒计时，由 REMOTEIO_UpdateStatus()
递减 */
} RC_ctrl_t;
```

遥控器共有8个通道，四个摇杆和四个拨杆，解码后的数值为-1024~+1024，回中值为0

如果回中值不为0一般有两个原因：

1. 不小心动了遥控器上的微调量，需要保证为0，如果不为0可以通过调整T1 T2 T3 T4来修改



2. 遥控器有零漂，手动调整头文件中的宏定义参数修改

```
20  /* -----遥控器各摇杆回中值----- */
21  /* 此处为原始数据的回中值 可以根据实际情况修改 */
22  #define RC_CH0_VALUE_OFFSET ((uint16_t)1024) //右横摇杆
23  #define RC_CH1_VALUE_OFFSET ((uint16_t)1024) //右竖摇杆
24  #define RC_CH2_VALUE_OFFSET ((uint16_t)1024+4) //左横摇杆
25  #define RC_CH3_VALUE_OFFSET ((uint16_t)1016) //左竖摇杆
```